

Analytics Drill-Down: Frontend Integration Guide (v2.0)

Hello Frontend Team! 🙌

To support multiple databases (GLIF, BANCS, P&L) and complex runtime calculations without requiring a frontend redeployment for every change, the Backend Architecture has been upgraded to an **Enterprise Semantic Layer**.

What this means for you:

1. **No More Hardcoding:** You must remove the `REPORT_TYPES` constant from your React code.
2. **Logical Names Only:** You will no longer deal with physical database columns. You will request "Logical" names (e.g., `TotalAmount` or `PnlCategory`), and the backend will securely translate those into complex Druid SQL behind the scenes.
3. **View Codes:** Every drill-down request now requires a `viewCode`. This tells the backend exactly which database to route the query to.

1. The Complete UI Workflow

Here is exactly how the React application should behave from load to click:

1. **On Component Mount:** Call `GET /api/v1/analytics/config`.
2. **Build the UI:** Use the response to populate your "Select Report" and "Select View" dropdowns.
3. **Initial Table Load:** When a user selects a view, take the `viewCode`, the first item in the hierarchy array, and the `logicalName` of the metrics, and send them to `POST /api/v1/analytics/drill-down` with an empty filters array.
4. **User Clicks [+]:** Append the current row's name to the filters array, grab the *next* item in the hierarchy array, and call the POST endpoint again.

2. API Contract 1: The UI Bootstrapper

This endpoint provides the permitted configurations for the currently logged-in user. It enforces Role-Based Access Control (RBAC), meaning a Branch user will automatically receive different views than an Admin.

- **Endpoint:** `GET /api/v1/analytics/config`
- **Headers:** `Authorization: Bearer <JWT_TOKEN>`

Expected Response

```
[
  {
    "reportCode": "PNL_REPORT",
    "displayName": "Profit & Loss Analysis",
```

```

    "views": [
      {
        "viewCode": "PNL_MAIN_VIEW",
        "displayName": "P&L Main Drill-down",
        "hierarchy": [
          "PnlCategory",
          "HeadCode",
          "GlCode"
        ],
        "availableMetrics": [
          {
            "logicalName": "TotalAmount",
            "displayName": "Total P&L Amount"
          }
        ]
      }
    ]
  }
}
]

```



Frontend Developer Action:

Use displayName for your UI labels. Store the viewCode, the hierarchy array, and the metric logicalNames in your React state, as you will need to pass them to the Drill-Down API.

3. API Contract 2: The Drill-Down Execution Engine

This is the workhorse endpoint. It fetches the actual numerical data from the Apache Druid cluster.

- **Endpoint:** POST /api/v1/analytics/drill-down
- **Headers:** Authorization: Bearer <JWT_TOKEN>, Content-Type: application/json

The Request Payload

Notice you are **no longer sending the full hierarchy array**. You only send the viewCode, the specific logical dimension you want right now, the metrics you want, and your current breadcrumb filters.

```

{
  "viewCode": "PNL_MAIN_VIEW",
  "dimensions": ["PnlCategory"],
  "metrics": ["TotalAmount"],
  "filters": []
}

```

Example of a Level 2 Request (User clicked "INCOME"):

```

{

```

```

    "viewCode": "PNL_MAIN_VIEW",
    "dimensions": ["HeadCode"],
    "metrics": ["TotalAmount"],
    "filters": [
      {
        "logicalField": "PnlCategory",
        "operator": "EQUALS",
        "value": "INCOME"
      }
    ]
  }
}

```

The Response Payload

The backend handles deterministic id generation and calculates hasChildren dynamically.

```

{
  "meta": {
    "status": "SUCCESS",
    "level": "PnlCategory",
    "rowCount": 2,
    "executionTimeMs": 42
  },
  "data": [
    {
      "id": "pnlcategory_income",
      "name": "INCOME",
      "TotalAmount": -123015480.84,
      "hasChildren": true
    },
    {
      "id": "pnlcategory_expense",
      "name": "EXPENSE",
      "TotalAmount": 50777230.42,
      "hasChildren": true
    }
  ]
}

```



Frontend Developer Action:

- **id:** Use this directly as your React key={row.id}. It guarantees the DOM will not flicker during state updates.
- **name:** This is the primary column value you render in the first cell of the row (e.g., "INCOME").
- **Metrics:** Note that the key returned (TotalAmount) perfectly matches the logicalName you requested in the payload.

- **hasChildren:** If true, render your [+] icon. If false, do not render an expansion icon.

4. Error Handling & Edge Cases

The backend will return standard HTTP status codes. Please handle them gracefully in the UI:

- **401 Unauthorized:** Missing or expired JWT token. Redirect to login.
- **403 Forbidden:** The user attempted to pass a viewCode they are not authorized to see (e.g., a Branch user trying to force an Admin view). Display a "Permission Denied" toast.
- **400 Bad Request:** You passed a dimension or metric that does not exist in the Oracle configuration for that specific view.
- **502 / 503 Bad Gateway:** The Apache Druid cluster is currently unreachable. Display a "Data Engine currently unavailable" message to the user.

Backend Engineering Team